

may be provided by the NLTK *downloader*. As you can see in Figure 1, NLTK Book is installed in my system. The following command will import all the downloaded books:

```
>>> from nltk.book import *
*** Introductory Examples for the NLTK Book ***
Loading text1, ..., text9 and sent1, ..., sent9
Type the name of the text or sentence to view it.
Type: 'texts()' or 'sents()' to list the materials.
text1: Moby Dick by Herman Melville 1851
text2: Sense and Sensibility by Jane Austen 1811
text3: The Book of Genesis
text4: Inaugural Address Corpus
text5: Chat Corpus
text6: Monty Python and the Holy Grail
text7: Wall Street Journal
text8: Personals Corpus
text9: The Man Who Was Thursday by G . K . Chesterton 1908
```

We can see that nine text books and nine sentences are imported with a single statement.

Creating NLTK Book

Users can create their own NLTK Book. The content of a text file, *a.txt*, is given below:

“Natural language processing (NLP) is an area of computer science and artificial intelligence concerned with the interactions between computers and human (natural) languages, in particular how to program computers to fruitfully process large amounts of natural language data. Challenges in natural language processing frequently involve speech recognition, natural language understanding and natural language generation.”

This can be generated as an NLTK text book using the statements given below:

```
>>> f1=open('a.txt','rU')
>>> txt=f1.read()
>>> token = nltk.word_tokenize(txt)
>>> nlp = nltk.Text(token)
>>> nlp
<Text: Natural-language processing ( NLP ) is an area...>
```

A text with the name 'nlp' is generated. Here, the file *a.txt* is opened in the *rU* mode, i.e., reading in universal newline mode. By opening a file in this mode, we make sure that newline characters in other platforms can also be supported by the platform on which Python is running. The use of the *word_tokenize()* function will be described later in this article.

The type of an NLTK Book can be obtained with the following statement:

```
>>>type(nlp)
```

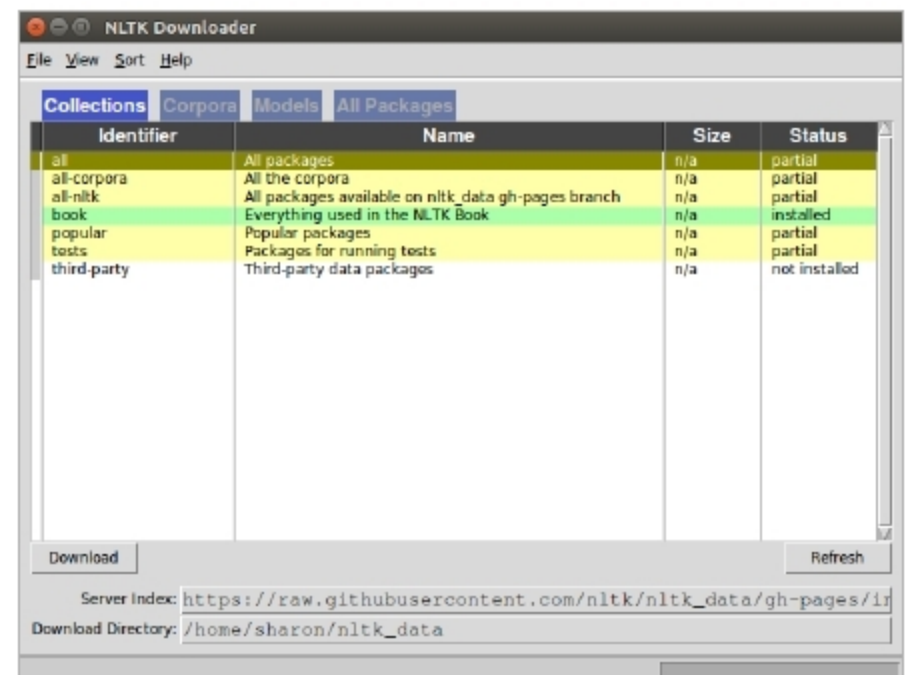


Figure 1: NLTK *downloader*

```
<class 'nltk.text.Text'>
```

One article is not enough to explain everything about NLTK. Some of the NLP tasks that can be performed with it are explained below.

Tokenisation

The input text needs to be broken down into basic linguistic units before applying some transformations. Tokenisation is the process of splitting a stream of characters into meaningful entities, called tokens. This splitting can be done sentence-wise or word-wise. A sentence boundary can be detected with the help of the *sent_tokenize* tool in NLTK. An example is shown below:

```
>>> a='hi sharon. how are you? revert me to sharon@gmail.com'
>>> nltk.sent_tokenize(a)
['hi sharon.', 'how are you?', 'revert me to sharon@gmail.com']
```

It is a sophisticated task to distinguish between the '.' (the full stop) at the end of sentences and the one that appears in the mail ID!

Word boundaries can be detected easily with the help of *word_tokenize()*. The result after word tokenisation of 'a' is given below:

```
>>> b=nltk.word_tokenize(a)
>>> b
['hi', 'sharon', '.', 'how', 'are', 'you', '?', 'revert', 'me', 'to', 'sharon', '@', 'gmail.com']
```

Parts of speech tagging

Identifying the parts of speech associated with each word is a natural language processing task. Once the input is tokenised, POS tagging can be done on it. If the input is not tokenised before POS tagging, each character in the input will be POS